

ISYS30221 Artificial Intelligence

Coursework Documentation

1- About this submission

Student Name	Rakhshan Yousaf
Student ID	N1230792
Chatbot Topic	Vehicles – Cars, Trucks, Motorcycles & Automotive Technology
Tasks implemented in this submission	☒ Task a ☒ Task b ☒ Task c ☐ Task a extra – short title: <u>Wikipedia fallback and spell correction</u> ☐ Task b extra – short title: _____ ☐ Task c extra – short title: _____
Files inventory (excluding this file)	1. mybot-basic.py: main chatbot program integrating Tasks A, B and C 2. mybot-basic.xml: AIML rule file for pattern-based conversation and command triggers 3. vehicles_qa.csv: CSV knowledge base containing question/answer pairs for similarity matching 4. vehicles_fol.csv: first-order logic knowledge base used by the reasoning component 5. train_vehicle_cnn.py: Python program used to train the image classification model 6. vehicle_classifier.h5: saved pre-trained CNN model for Task C 7. chat_log.txt: conversation log file generated by the chatbot 8. 3 sample vehicle images: test images used to demonstrate image classification (task c)
Demo video URL	Recording-20260327_110023.webm
Checklist	☒ I will submit this file separately (without compression) into DropBox ☒ All other files are zipped and will be submitted into DropBox ☒ The demo video is recorded as instructed, and the sharing link is inserted above ☒ I have made sure that the demo video is shared according to the instructions, so that I allowed everybody in the university to view it. ☒ All the sections below are populated accordingly.

2- Design notes (shrink/grow as needed, add images where applicable)

General explanations of the system and its goals	VehicleBot is a Python-based intelligent chatbot designed to answer questions about vehicles and automotive technology. The chatbot combines three different AI techniques in one integrated system. Its overall goal is to provide a smarter user interaction than a basic rule-based chatbot by supporting conversational responses, logical reasoning, and image classification. For Task A, the chatbot supports both pattern-based conversation and similarity-based question answering. AIML is used to provide direct responses for common vehicle-related questions such as greetings, car types, electric vehicles, safety features and maintenance topics. If the user enters a question that does not match an AIML rule, the chatbot falls back to a CSV question/answer knowledge base. It searches for the closest question using lemmatisation, TF-IDF vectorisation and cosine similarity, and then returns the most relevant answer. As an additional enhancement, the chatbot also includes simple spelling correction and a Wikipedia fallback for some unknown queries. For Task B, the chatbot includes a first-order logic knowledge base about vehicles. The user can add facts using the pattern "I know that ... is ..." and test facts using "Check that ... is ...". The logic component uses NLTK's resolution prover to decide whether a statement is true, false, or unknown based on the loaded facts and inference rules. This allows the chatbot to perform simple reasoning rather than only returning stored text answers. For Task C, the chatbot can classify an uploaded image as a car, motorcycle, or truck using a pre-trained convolutional neural network (CNN). When the user enters an image command, the chatbot opens a file dialog box, loads the chosen image, processes it and predicts the most likely vehicle class with a confidence percentage. The system was designed to show how different AI techniques can be combined into one application. The chatbot therefore acts as a rule-based conversational agent, a similarity-based retrieval system, a logic-based reasoning agent, and an image classification assistant in a single program.
The system requirements, i.e., the list of what the system should do/have from a user's perspective	From the user's perspective, the system is required to: • allow the user to have a text-based conversation about vehicles and automotive topics. • respond to basic vehicle questions using AIML pattern matching. • answer vehicle questions that are not exact AIML matches by finding the closest question in a CSV knowledge base. • use lemmatisation, TF-IDF and cosine similarity for this similarity-based matching process. • recognise and correct some common spelling mistakes in vehicle-related words. • provide a fallback answer using Wikipedia when no AIML or similarity answer is found. • allow the user to add knowledge in the form "I know that X is Y".

	- allow the user to query the logic knowledge base using “Check that X is Y”. - return one of the following logic responses where appropriate: Correct, Incorrect, or I don’t know. - load a pre-trained CNN model for image classification. - allow the user to select an image file from the computer. - classify the selected image into one of the supported vehicles classes; - display the predicted class and confidence score. - log the conversation between the user and the chatbot to a text file. - allow the user to end the conversation using an exit command such as bye or exit.
The employed AI techniques, and the explanation of program codes and the supplied files.	The chatbot uses several AI techniques across the three tasks. Task A: AIML, NLP preprocessing and similarity search: For rule-based conversation, the chatbot uses the AIML library. The file mybot-basic.xml contains patterns and templates for greetings, identity questions, vehicle definitions, electric vehicles, hybrid cars, safety features, car types, maintenance and other automotive topics. Some AIML rules also trigger Python commands such as exiting the program, calling the logic module, calling the image classifier, or using Wikipedia fallback. For similarity-based conversation, the chatbot loads question/answer pairs from vehicles_qa.csv. Each question is pre-processed using tokenisation, lowercasing and lemmatisation with NLTK’s WordNetLemmatizer. After preprocessing, the questions are converted into TF-IDF vectors using TfidfVectorizer from scikit-learn. When the user asks a question that does not match AIML, the chatbot transforms the input into a TF-IDF vector and calculates cosine similarity against all stored questions. The answer linked to the highest-scoring question is returned if the similarity is above a threshold. As additional Task A functionality, the chatbot includes a simple spelling correction dictionary for common vehicle-related misspellings such as “eletric”, “vehical”, “battry” and “insurence”. It also includes a Wikipedia search fallback for some unknown questions. The chatbot also saves each conversation turn into chat_log.txt. Task B: First-order logic and inference: The logic component is implemented using NLTK’s logic and inference tools. The file vehicles_fol.csv contains facts and rules written in first-order logic syntax, for example Electric(Tesla) and all x. (Electric(x) -> Vehicle(x)). When the chatbot starts, these statements are loaded and converted into NLTK logic expressions. The function process_logic_input() converts user inputs such as “I know that Tesla is Electric” into FOL form Electric(Tesla). For ADD mode, the chatbot checks whether the new statement already follows from the current knowledge base or contradicts it. If there is no contradiction, the fact is added to the in-memory knowledge base. For CHECK mode, the chatbot uses ResolutionProver to test whether the statement is provable, whether its negation is provable, or whether

neither can be proved. Based on this, it returns **Correct, Incorrect, or Sorry, I don't know if that is true based on my current knowledge.**

Task C: CNN image classification:

The image classification component uses **TensorFlow/Keras**. A CNN model was trained separately in `train_vehicle_cnn.py` and saved as `vehicle_classifier.h5`. In the main chatbot, the model is loaded at startup using `load_model()`. When the user enters a supported image command, the program opens a Tkinter file selection dialog. The selected image is resized to 128 x 128, converted into an array, normalised, and then passed to the CNN model for prediction.

The predicted probability scores are used to choose one of the supported class labels:

- car
- motorcycle
- truck

The chatbot then returns a sentence such as **"I am 87.9% sure that is a truck."**

Explanation of the main program structure:

The main program file is `mybot-basic.py`. It performs the following major steps:

1. imports the required libraries for AIML, NLP, logic, machine learning and GUI file selection;
2. loads and preprocesses the CSV Q/A knowledge base for Task A;
3. loads the first-order logic knowledge base for Task B;
4. loads the trained CNN model for Task C;
5. bootstraps the AIML kernel using `mybot-basic.xml`;
6. enters a continuous chat loop where each user input is:
 - spell-corrected,
 - checked against AIML,
 - routed to logic or image processing if needed,
 - otherwise passed to similarity search and then Wikipedia fallback;
7. logs the final response to `chat_log.txt`.

This design keeps all three tasks integrated into a single chatbot system as required by the coursework brief.

3- Conversation log (insert text, screenshots and/or images as required)

Task A: Conversational capabilities:

A1. AIML pattern-based conversation

> hello
Hello! I am VehicleBot. Ask me anything about cars, EVs, or automotive technology.

> what is your name
I am VehicleBot, your friendly automotive assistant built for the ISYS30221 coursework!

> what can you do
I can answer questions about vehicles, electric cars, car types, engines, safety features, and more. I also have a knowledge base and can search Wikipedia. Try asking me something!

> what is torque
Torque is the rotational force produced by the engine, measured in Newton-metres (Nm). It determines how quickly a vehicle can accelerate, especially from a standstill.

> what is MPG
MPG stands for Miles Per Gallon. It measures fuel efficiency. Higher MPG means better fuel economy and lower fuel costs.

These examples were answered directly by AIML pattern matching. The user input matched predefined categories in mybot-basic.xml, so the chatbot returned a rule-based response without needing similarity search.

> What is fuel economy?
[Similarity match – score: 1.00]
Fuel economy measures how efficiently a vehicle uses fuel, often in miles per gallon or litres per 100 km.

> What is vehicle insurance?
[Similarity match – score: 1.00]
Vehicle insurance is a contract that covers financial loss if your car is damaged, stolen or causes injury or damage.

> What are run flat tyres?
Run-flat tyres can continue driving at reduced speed (up to 50 miles) after a puncture, so you can safely reach a garage without changing the tyre on the roadside.

> What is lane assist?
Lane assist uses cameras to detect lane markings and gently steers the car back into the lane if you start to drift without indicating.

> What is fuel economy in cars?
[Similarity match – score: 0.79]
Fuel economy measures how efficiently a vehicle uses fuel, often in miles per gallon or litres per 100 km.

These examples demonstrate the similarity-based fallback. When the user input did not trigger a direct AIML response, the chatbot preprocessed the sentence using lemmatisation, transformed it into TF-IDF form, and selected the closest matching CSV question using cosine similarity.

In some cases the similarity system returned less relevant answers because lexical overlap was stronger with another stored question. This is a limitation of simple TF-IDF matching on a relatively small domain-specific dataset.

Task B: Rule-based logical reasoning:

```
> Check that Tesla is Electric
[Spell-check corrected your input to: "Check that Tesla is Electric"]
Correct. That is true.
```

```
> Check that Tesla is Vehicle
[Spell-check corrected your input to: "Check that Tesla is Vehicle"]
Correct. That is true.
```

This output is inferred using the rule $\forall x. (\text{Electric}(x) \rightarrow \text{Vehicle}(x))$ in the logic knowledge base.

```
> Check that Tesla is Petrol
[Spell-check corrected your input to: "Check that Tesla is Petrol"]
Incorrect. That is false.
```

This is inferred using the rule $\forall x. (\text{Electric}(x) \rightarrow \neg \text{Petrol}(x))$. Since Tesla is electric, the system can prove that Tesla is not petrol.

```
> Check that Bugatti is Expensive
[Spell-check corrected your input to: "Check that Bugatti is Expensive"]
Correct. That is true.
```

This follows from the fact $\text{Fast}(\text{Bugatti})$ and the rule $\forall x. (\text{Fast}(x) \rightarrow \text{Expensive}(x))$.

```
> I know that NissanLeaf is Electric
OK, I will remember that NissanLeaf is Electric.
```

```
> Check that NissanLeaf is Electric
Correct. That is true.
```

```
> Check that NissanLeaf is Vehicle
Correct. That is true.
```

After adding the new fact, the system infers that NissanLeaf is also a vehicle using the electric-to-vehicle rule.

```
> Check that NissanLeaf is Petrol
Incorrect. That is false.
```

```
> Check that BMW is Vehicle
[Spell-check corrected your input to: "Check that BMW is Vehicle"]
Sorry, I don't know if that is true based on my current knowledge.
```

The chatbot knows $\text{German}(\text{BMW})$ but there is no rule linking $\text{German}(x)$ to $\text{Vehicle}(x)$, so the statement is treated as unknown rather than false.

Task C: Image classification:

```
> What is in this image  
[Opening file selector...]  
I am 87.9% sure that is a truck.
```

```
> what is in this image  
[Opening file selector...]  
I am 99.9% sure that is a motorcycle.
```

```
> What is in this image  
[Opening file selector...]  
I am 80.2% sure that is a car.
```

These examples demonstrate the Task C image classification feature. The chatbot triggers the CNN model through an AIML command, opens a file selection window, preprocesses the selected image, and predicts whether the image is a car, motorcycle or truck. The confidence score is the highest SoftMax probability returned by the model. There were slightly more things I can have implemented in task b and task c but I left the coursework really late. Overall the tasks went well.

Performance and limitations: The CNN model performed well on clear single-object vehicle images, especially when the vehicle was centred and visually similar to the training data. However, confidence was lower for some images, especially when the image quality, angle, background or lighting differed from the training set. This is expected because the model was trained on a limited number of classes and may not generalise perfectly to all real-world images.

